SWINBURNE UNIVERSITY OF TECHNOLOGY

COS30082
Applied Machine Learning

Lecture 3

Deep Neural Networks

Why non-linear hypothesis?

Binary classification : Dog or Cat













Why non-linear hypothesis?

Binary classification : Dog or Cat

Features: colour/pixel intensity

Classification model

Testing phase

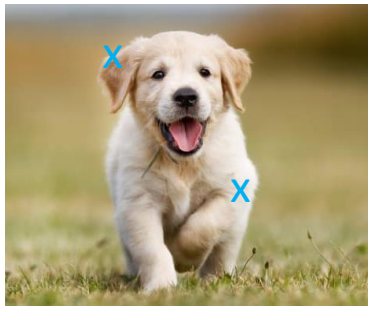
Test image

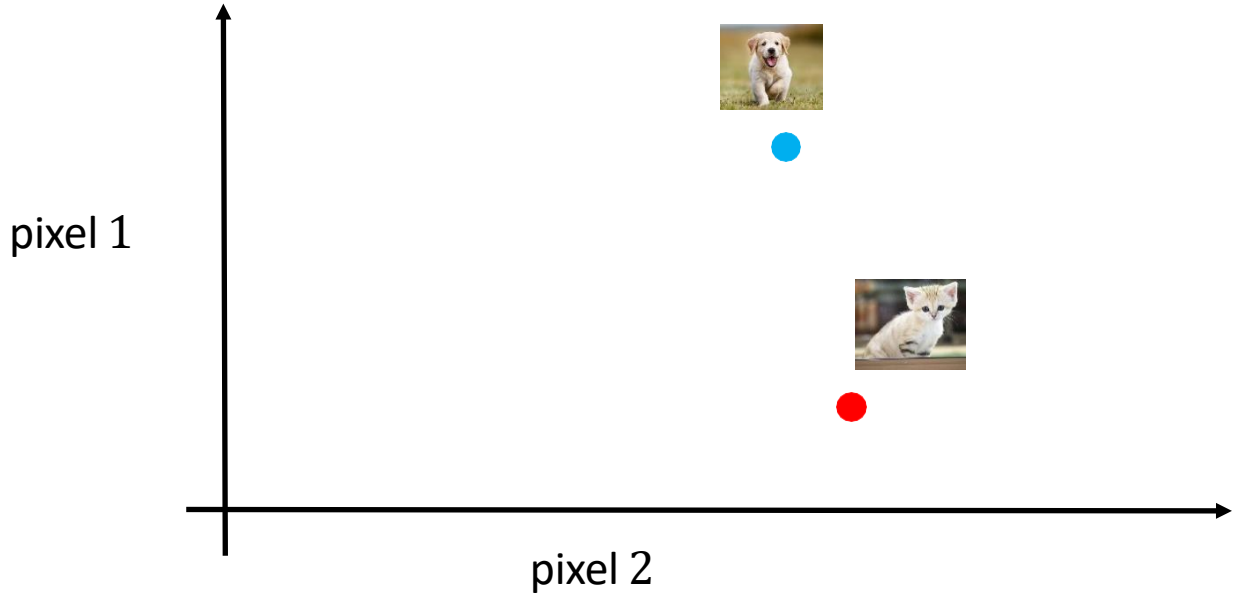
Features: colour/pixel intensity

Classification model

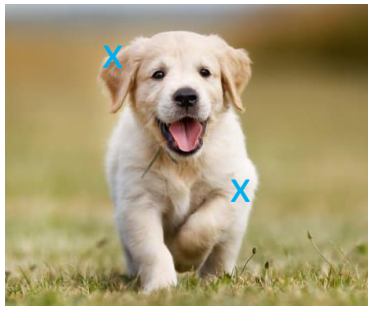
Predicted as dog

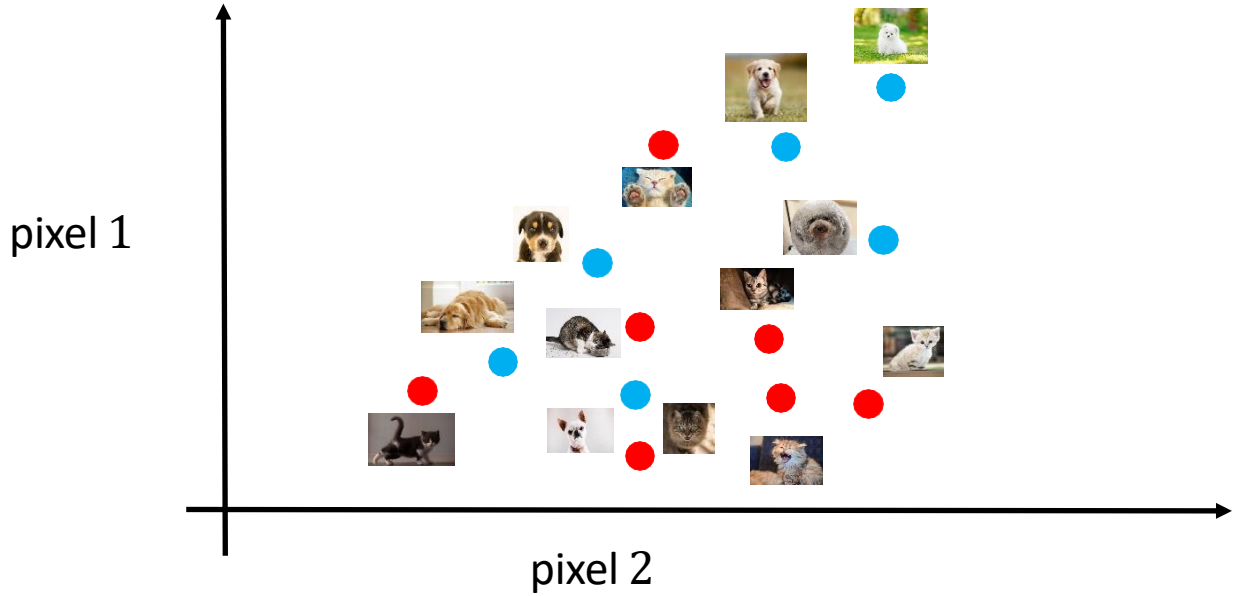
Why non-linear hypothesis?



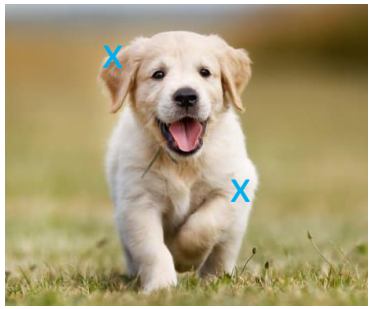


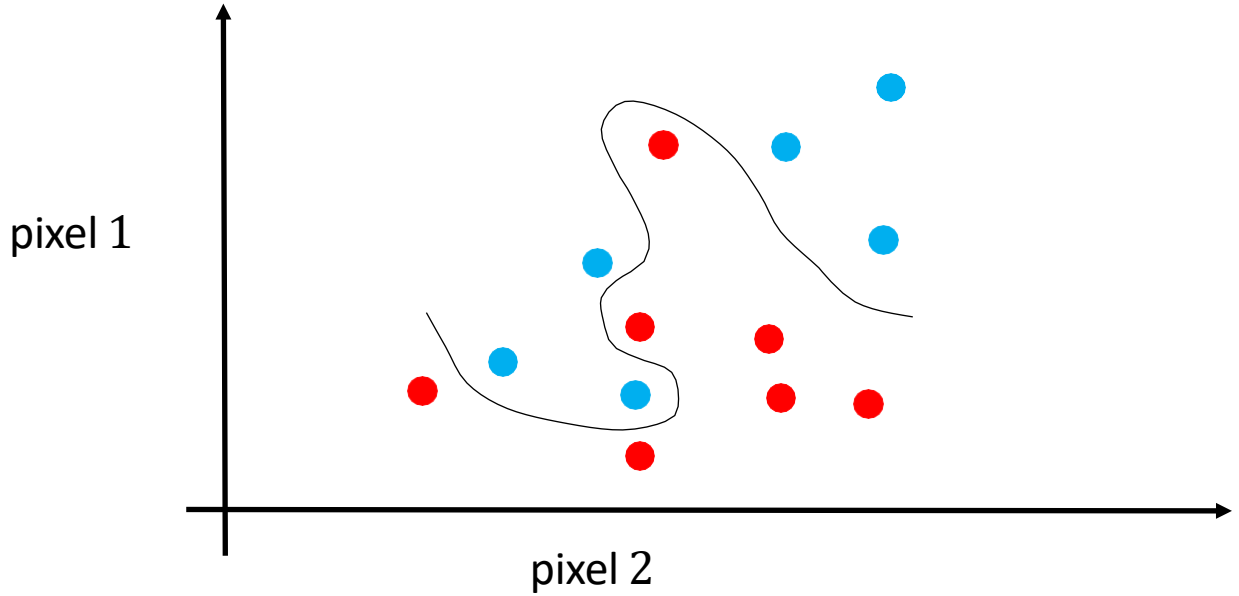
Why non-linear hypothesis?



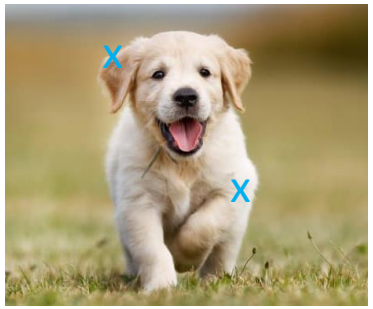


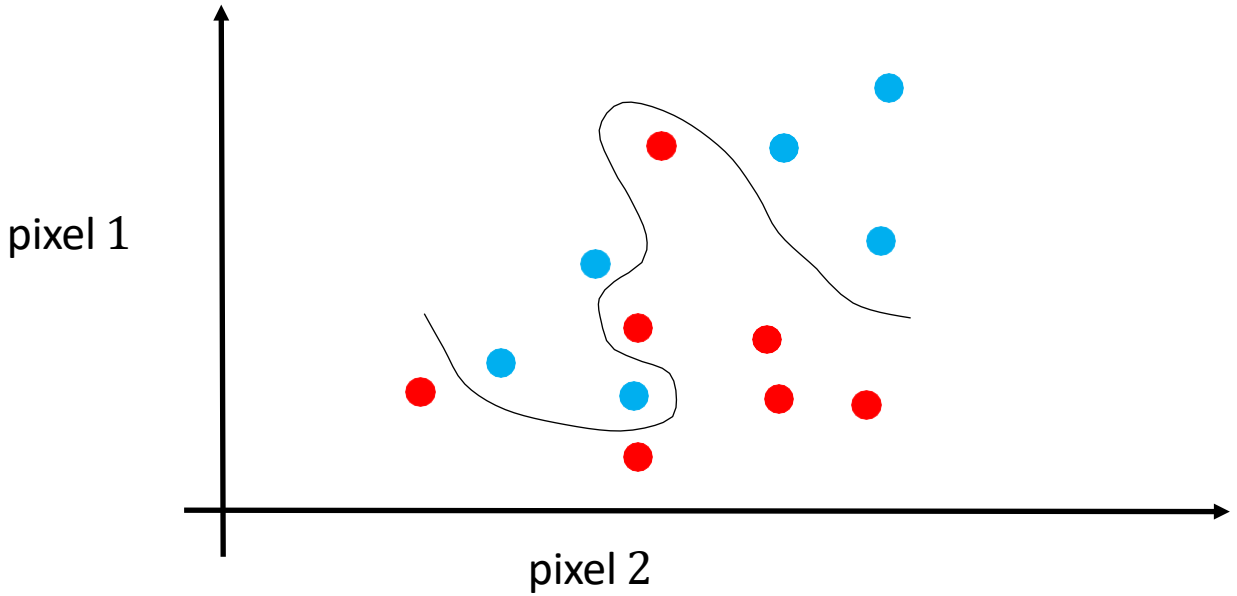
Why non-linear hypothesis?





Why non-linear hypothesis?

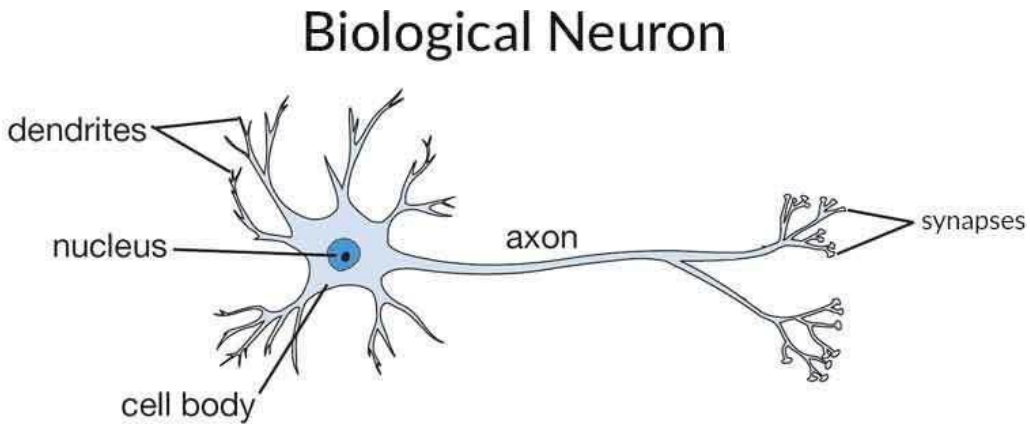




- Imagine, the size of the features is the image size: n = 60 x 60 pixels
- Hence, we have 3600 number of features per image (10800 for RGB images).
- If we want to achieve non linear hypothesis and will to feed in *quadratic features* (x^2), then the $n \approx 58$ million.
- Features that are too large are not a good way of representation and lead to high computational cost.
- Neural network provides a better alternative to learn non-linear model.

Understanding of ANN

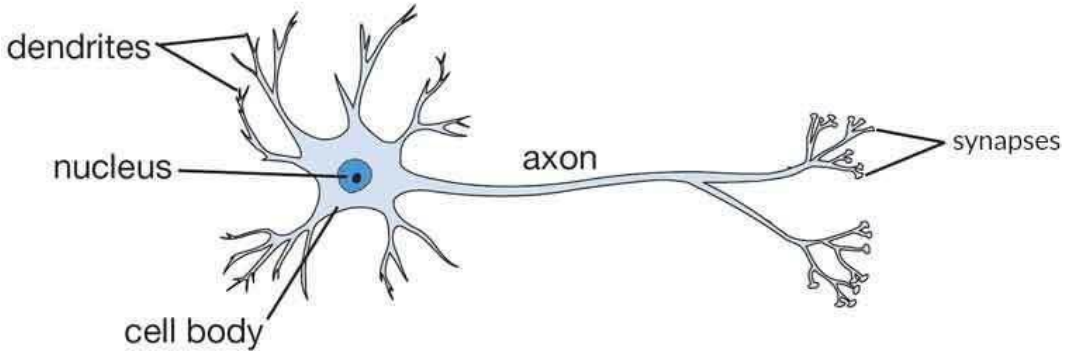
- The human brain has hundreds of billions of cells called neurons. Each neuron is made up of a cell body that is responsible for processing information by transporting information to (inputs) and from (outputs) from the brain.

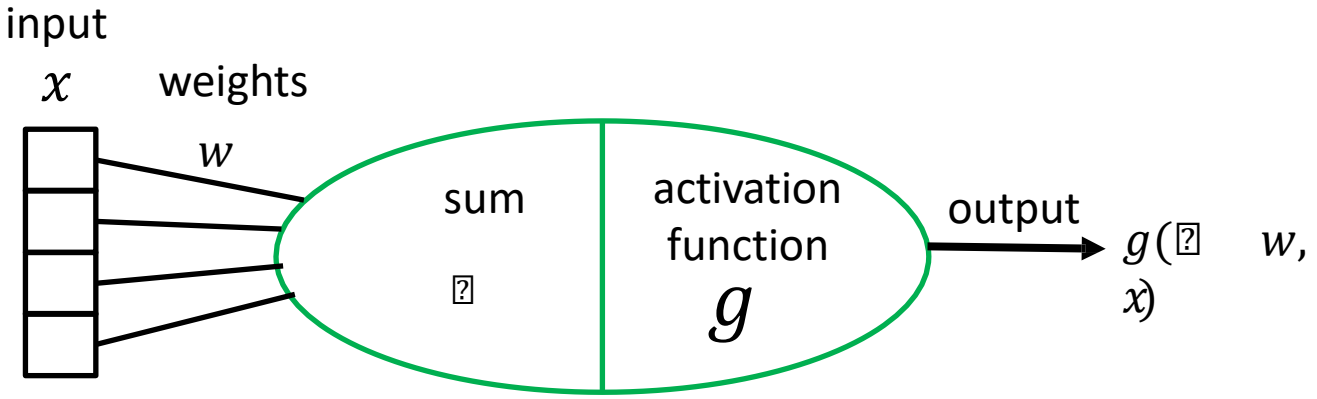


https://www.xenonstack.com/blog/artificial-neural-network-applications/

Understanding of ANN

- Artificial neural networks are built like the human brain. It has hundreds or thousands of artificial neurons which are interconnected. The input neurons receive various forms of data and, based on an internal weighting system, the neural network learns the patterns of the data and matches them to the target output.





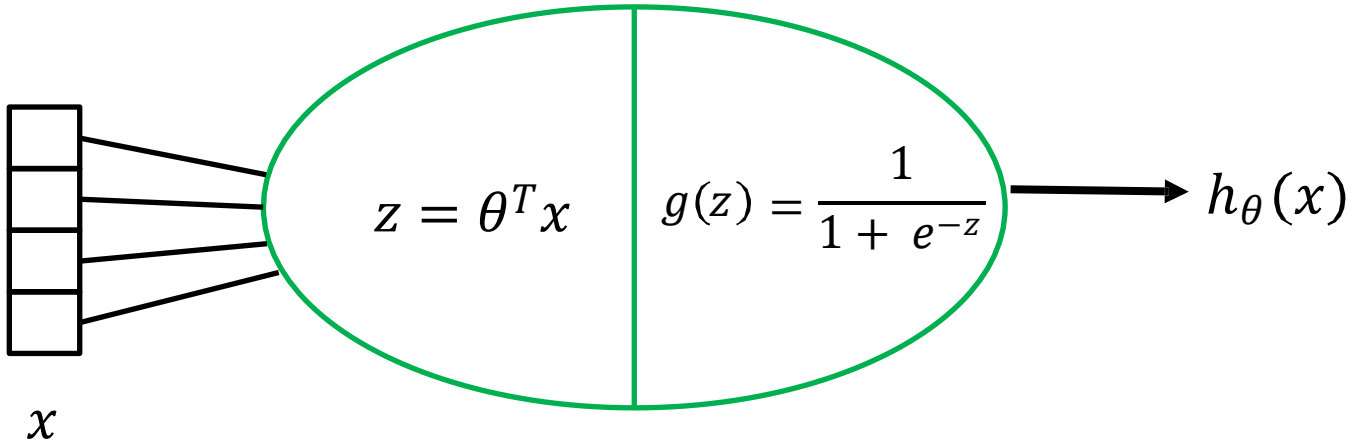
Neuron model for logistic regression

- In binary classification, to ensure predicted value $h_\theta\,(x)$ lies between 0 and 1:

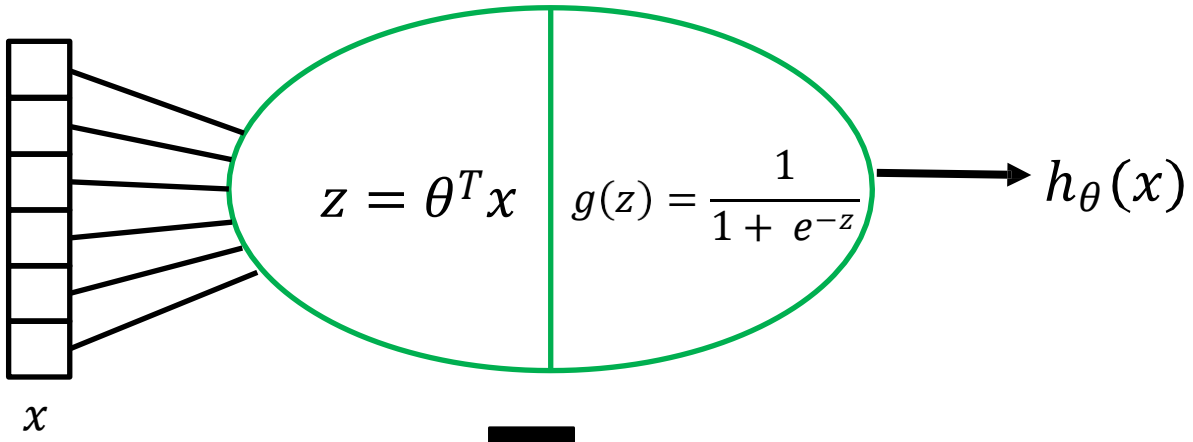
$$1 \leq h_\theta(x) \leq 0$$

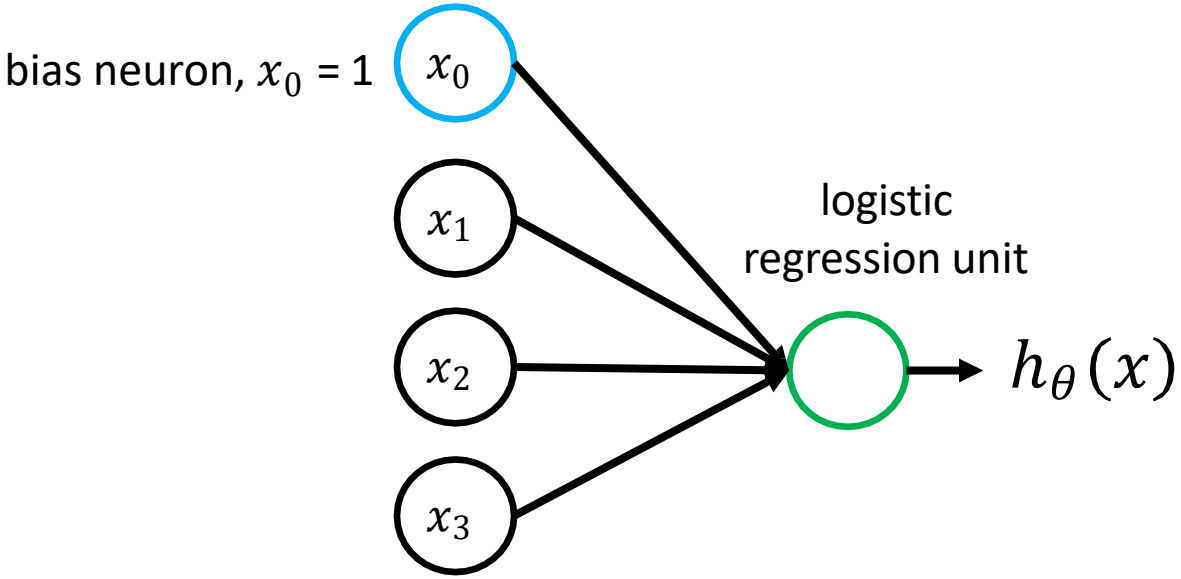
$h_\theta(x)$ is represented as $\rightarrow h_\theta\,(x) = g(z)$

where $z = \theta^T x \quad g(z) = \frac{1}{1+e^{-z}}$



Neuron model for logistic regression





- Says an input x with 3 dimensional features
- The neuron model terminologies for LR:

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$ input

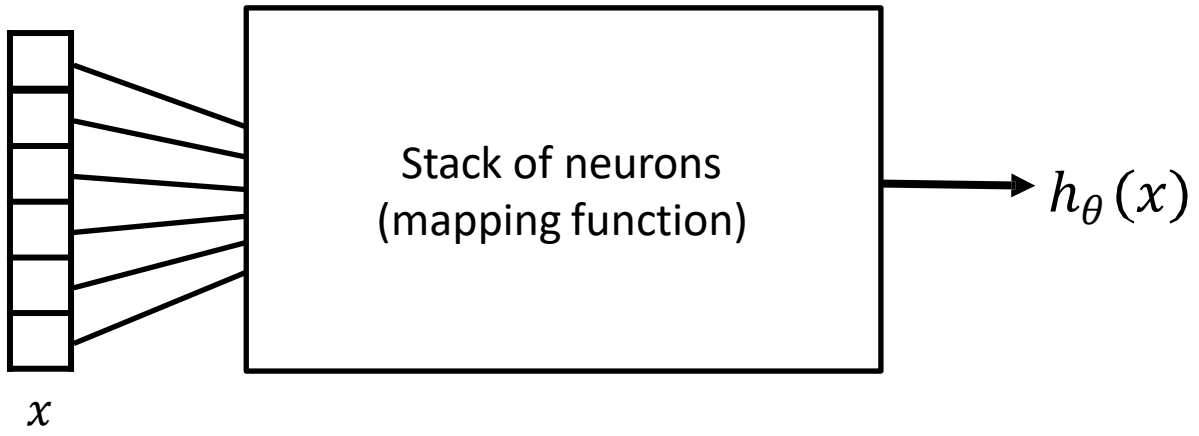
$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$ weights / parameters

$g(z)$ sigmoid (regression) activation function

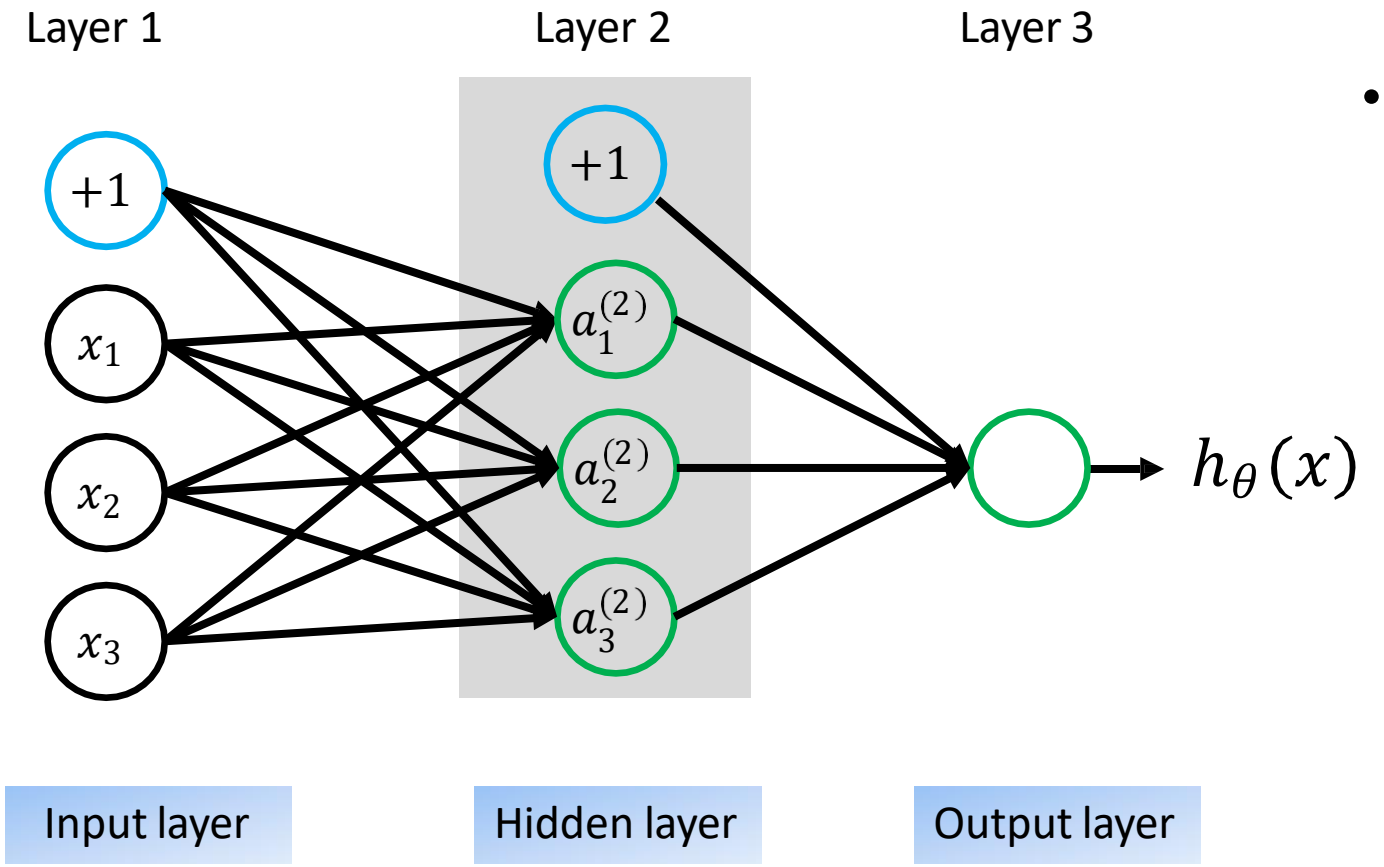
$h_\theta(x)$ predicted output

Neural network

- Neural network consists of stack of neurons that takes in input x and output predicted value $h_\theta(x)$
- Given enough number of neurons, neural network are incredibly good at mapping input x to the actual output y.



Neural network (single hidden layer)



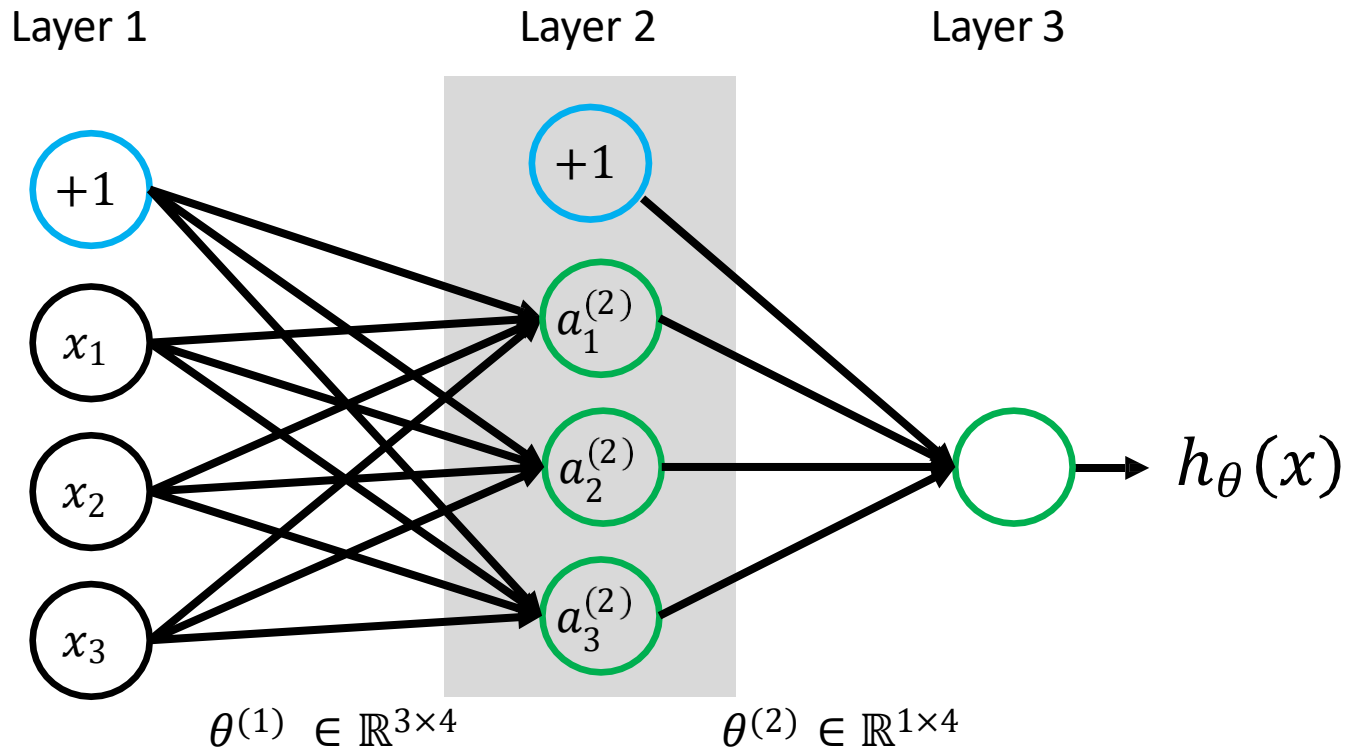
- Notation used in computation:

  $a_i^{(j)}$: activation of neuron i in layer j

  θ^j: matrix of weights make up the mapping function from layer j to $j+1$

  s_j: number of neurons in layer j not including bias

Neural network (single hidden layer)



- Notation used in computation:

$a_i^{(j)}$: activation of neuron i in layer j

θ^j: matrix of weights make up the mapping function from layer j to $j+1$

s_j: number of neurons in layer j not including bias

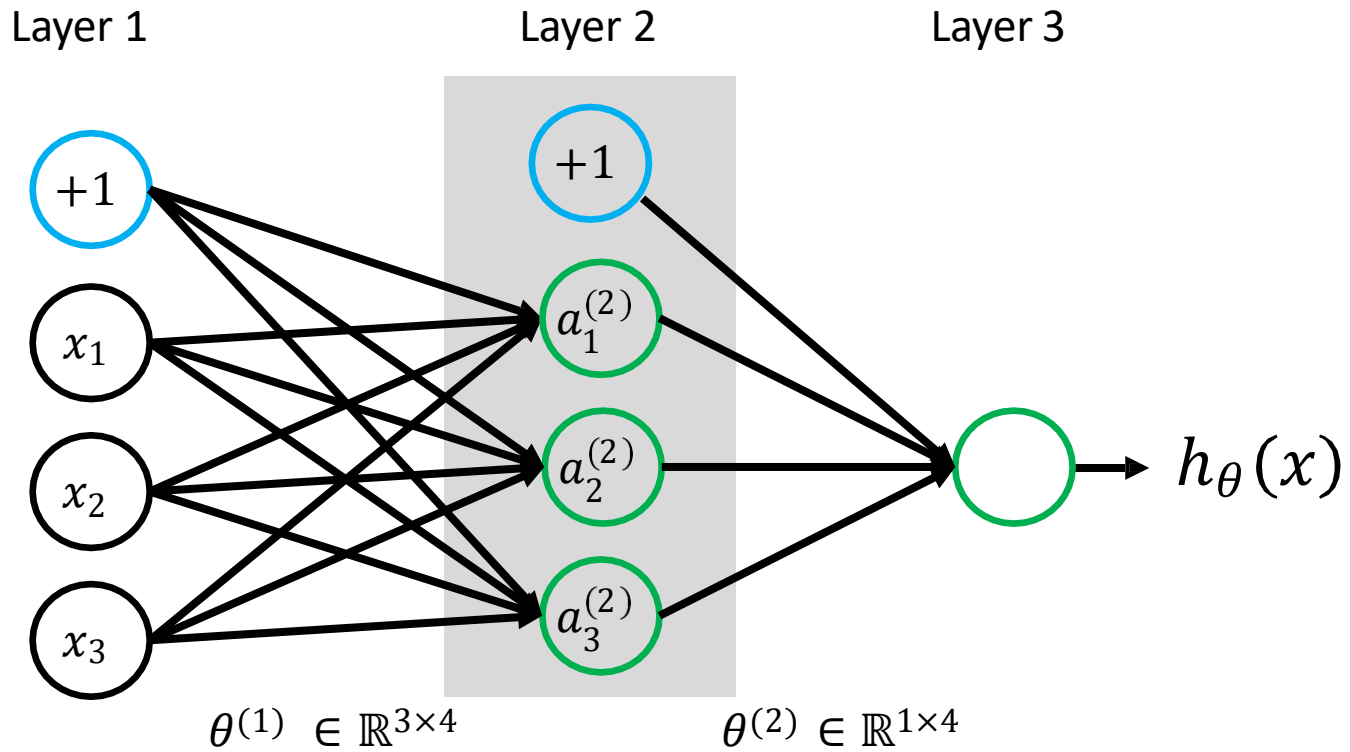
$$a_1^{(2)} = g(\theta_{10}^{(1)}\cancel{x_o}^{1} + \theta_{11}^{(1)}x_1 + \theta_{12}^{(1)}x_2 + \theta_{13}^{(1)}x_3)$$
$$a_2^{(2)} = g(\theta_{20}^{(1)}x_o + \theta_{21}^{(1)}x_1 + \theta_{22}^{(1)}x_2 + \theta_{23}^{(1)}x_3)$$
$$a_3^{(2)} = g(\theta_{30}^{(1)}x_o + \theta_{31}^{(1)}x_1 + \theta_{32}^{(1)}x_2 + \theta_{33}^{(1)}x_3)$$
$$h_\theta(x) = a_1^{(3)} = g(\theta_{10}^{(2)}\cancel{a_0^{(2)}}^{1} + \theta_{11}^{(2)}a_1^{(2)} + \theta_{12}^{(2)}a_2^{(2)} + \theta_{13}^{(2)}a_3^{(2)})$$

Matrix format:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Neural network (single hidden layer)



$$a_1^{(2)} = g(\theta_{10}^{(1)}\cancel{x_o}^{1} + \theta_{11}^{(1)}x_1 + \theta_{12}^{(1)}x_2 + \theta_{13}^{(1)}x_3)$$
$$a_2^{(2)} = g(\theta_{20}^{(1)}x_o + \theta_{21}^{(1)}x_1 + \theta_{22}^{(1)}x_2 + \theta_{23}^{(1)}x_3)$$
$$a_3^{(2)} = g(\theta_{30}^{(1)}x_o + \theta_{31}^{(1)}x_1 + \theta_{32}^{(1)}x_2 + \theta_{33}^{(1)}x_3)$$
$$h_\theta(x) = a_1^{(3)} = g(\theta_{10}^{(2)}\cancel{a_0^{(2)}}^{1} + \theta_{11}^{(2)}a_1^{(2)} + \theta_{12}^{(2)}a_2^{(2)} + \theta_{13}^{(2)}a_3^{(2)})$$

- Notation used in computation:

  $a_i^{(j)}$: activation of neuron i in layer j

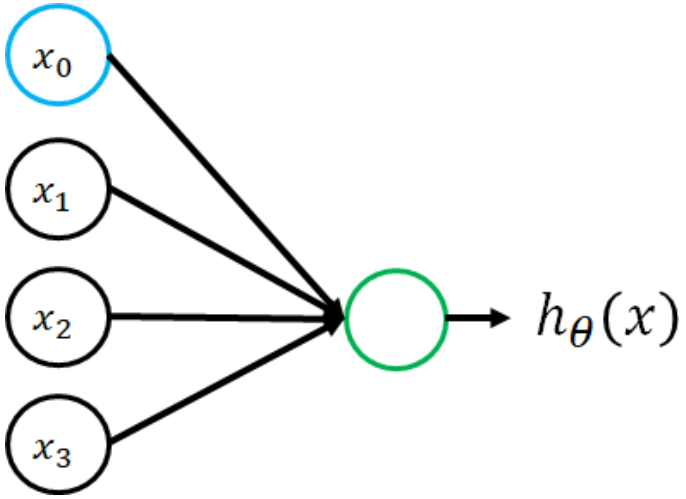
  θ^j: matrix of weights make up the mapping function from layer j to $j+1$

  s_j: number of neurons in layer j not including bias

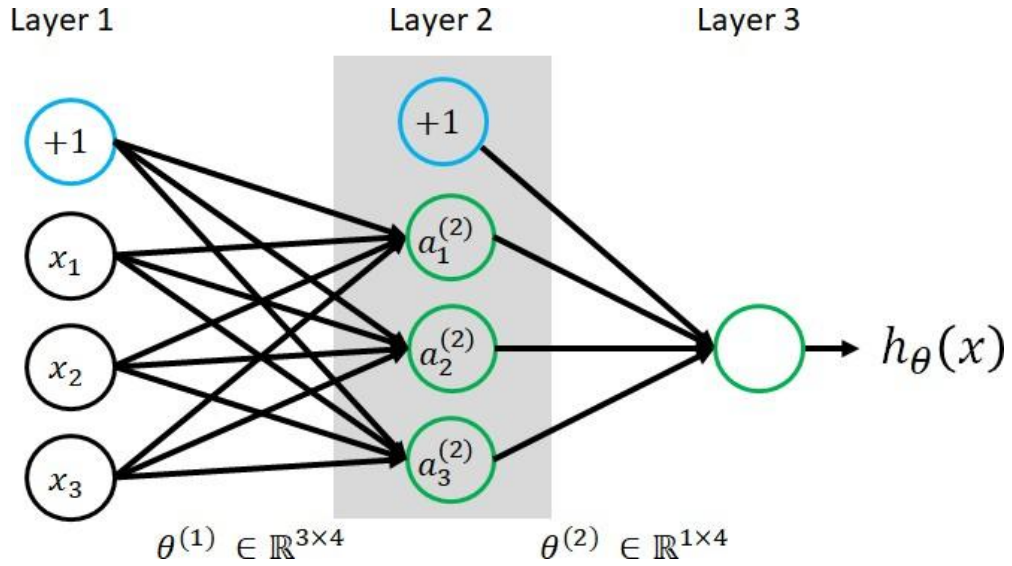
- Note that, if the network has s_j neurons in layer j and s_{j+1} neurons in layer $j+1$, then, the matrix dimension for θ^j is $s_{j+1} \times (s_j+1)$

Feature learning capability

Logistic regression



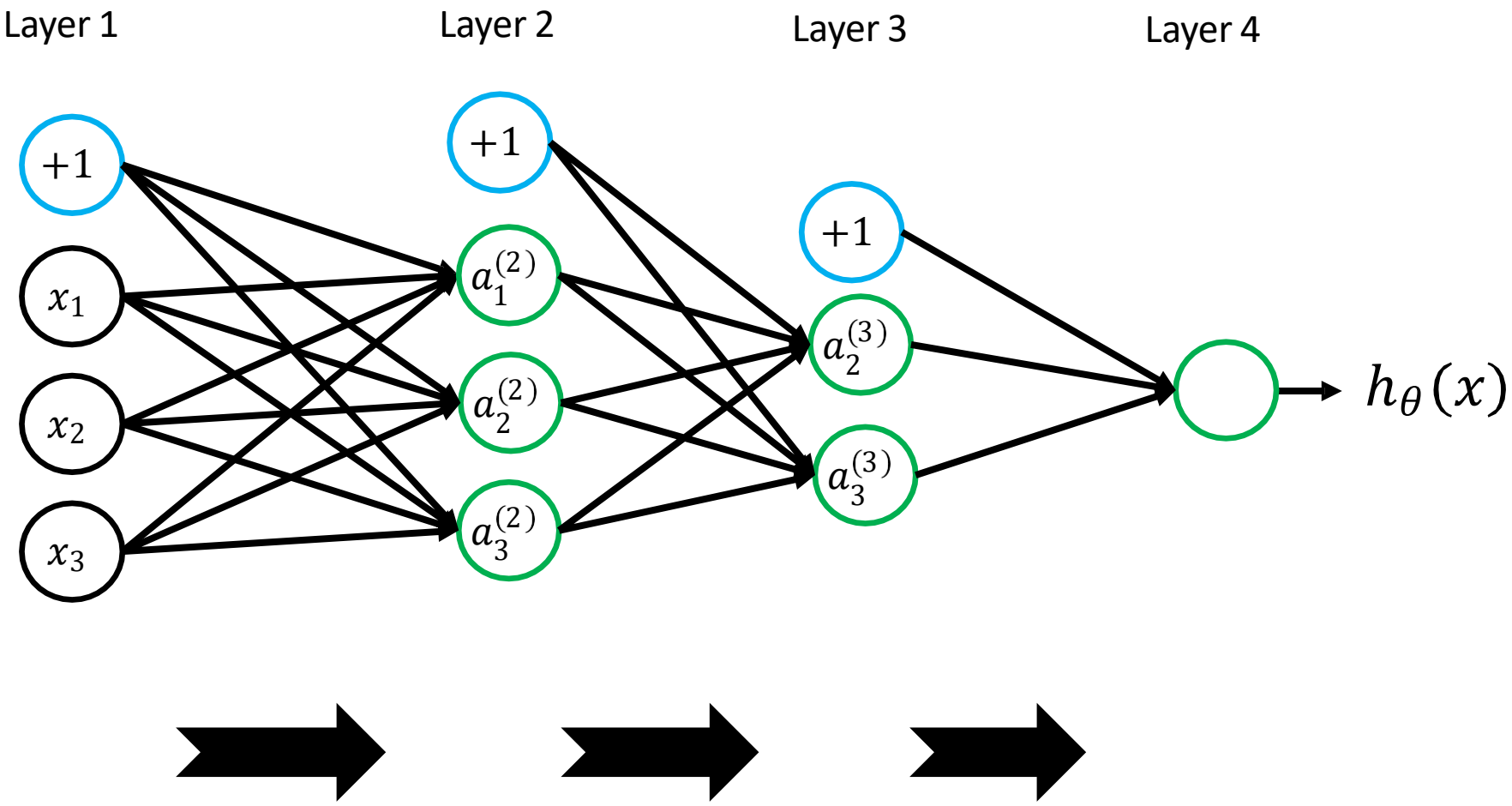
Neural Networks



- Logistic regression can be considered as a neural network without a hidden layer. Its mapping function connects the input x to the output y by a sigmoid function of a linear combination of features.

- A neural network is **more complex** than logistic regression. Normally, it has at least one hidden layer, which consists of the new features ($a^{(2)}$) that are learned as a function of $f(\theta^{(1)}, x)$.
  - With such flexibility in feature learning, neural networks can learn more complex features and output a more complex non-linear hypothesis.

Neural network with deeper layers



The complexity of the features increases

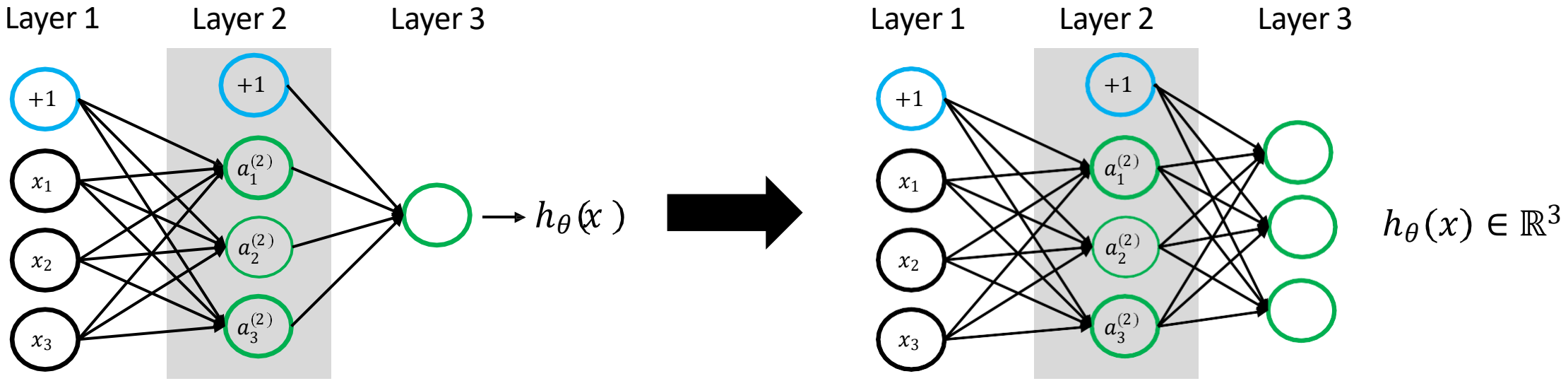
Multiclass classification

Class 1: dog

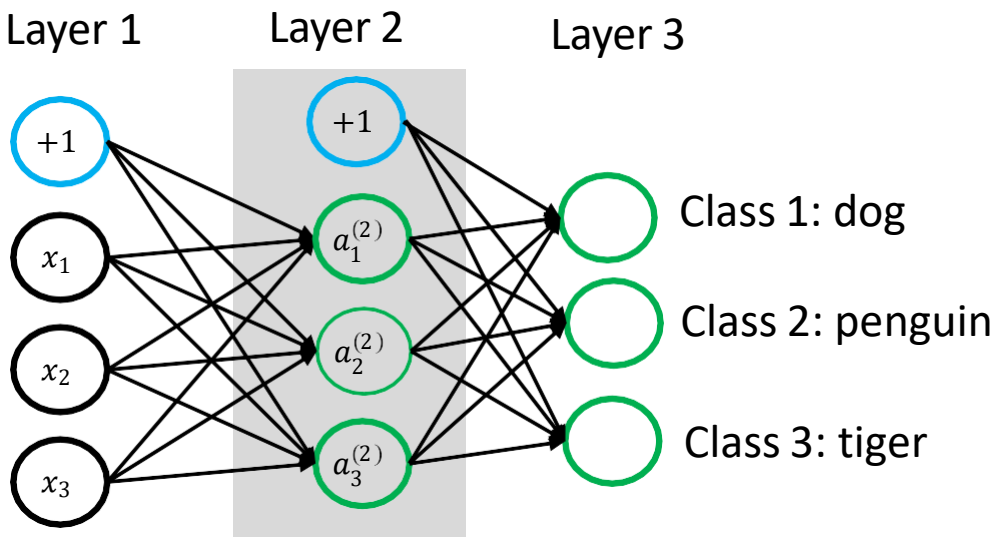
Class 2: penguin

Class 3: tiger

Multiple output neurons



Multiple output neurons



- Given *N* number of data points (training set) = $(x^{(1)}, y^{(1)})$, $(x^{(2)}, y^{(2)})$,……, $(x^{(N)}, y^{(N)})$
- We no longer label $y^{(n)} \in \{1,2,3\}$ but $y^{(n)} \in \mathbb{R}^3$ as follows:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Class 1: dog

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Class 2: penguin

$$y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Class 3: tiger

Multiple output neurons

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

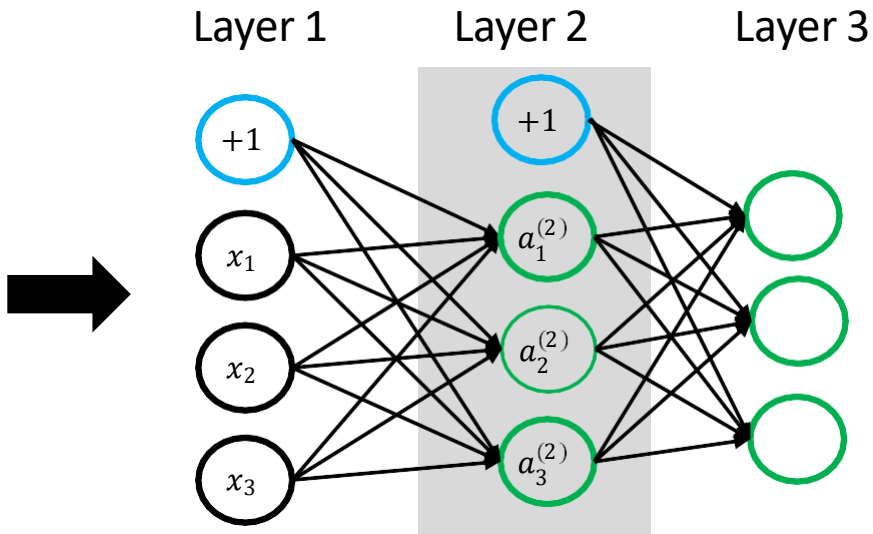
Class 1: dog

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Class 2: penguin

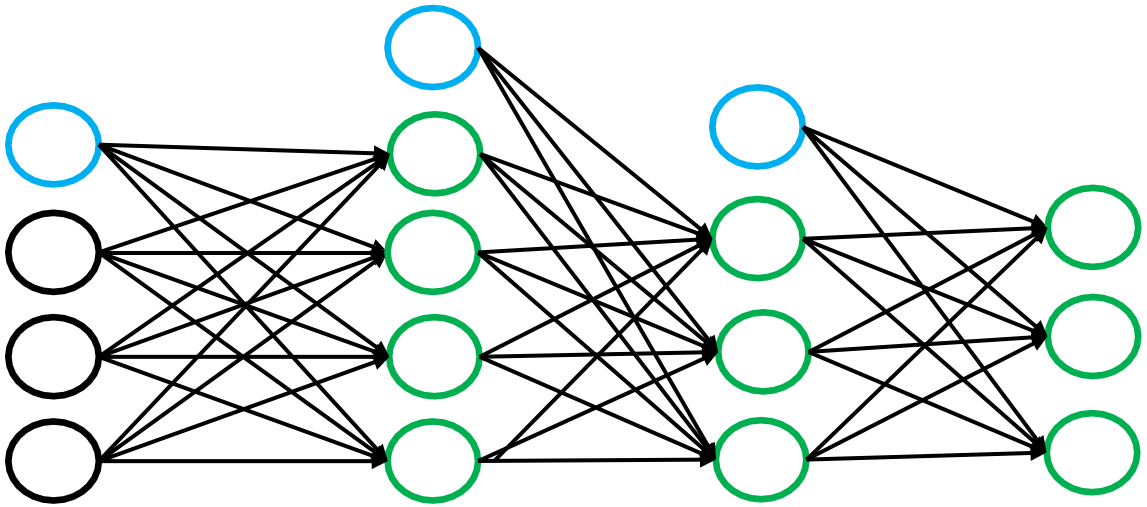
$$y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Class 3: tiger



$$\rightarrow h_\theta(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Cost function (Classification)



$K = 3$
$L = 4$
$s_1 = 3 \quad s_2 = 4 \quad s_3 = 3 \quad s_L = 3$
$y \in \mathbb{R}^3$
$h_\theta(x) \in \mathbb{R}^3$

- Terminology used:

K : number of output neurons
L : total number of layers in network
s_l: Number of neurons in layer l (not including bias)

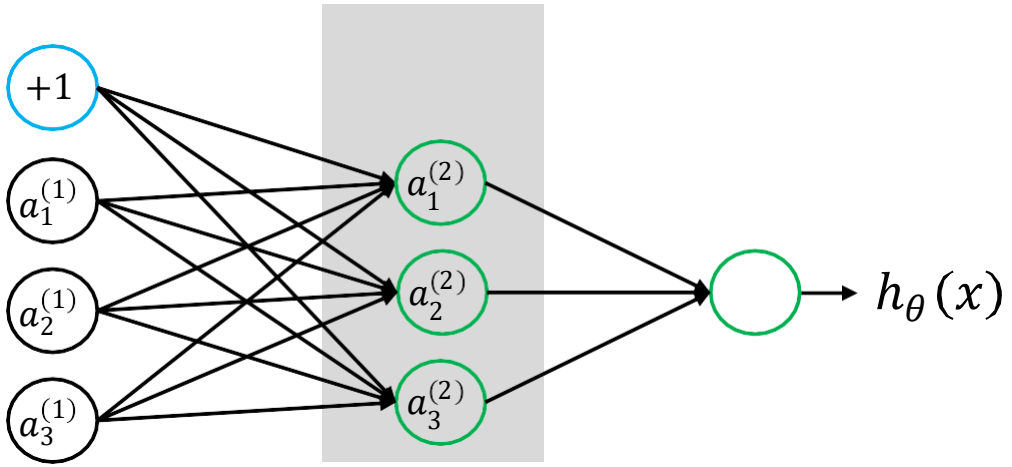
- For multiclass classification (K classes where $K \geq 3$)

$$y \in \mathbb{R}^K$$
$$h_\theta(x) \in \mathbb{R}^K$$

- For binary classification:

$$y \in \{0,1\}$$
$$h_\theta(x) \in \mathbb{R}$$

Gradient computation - forward propagation

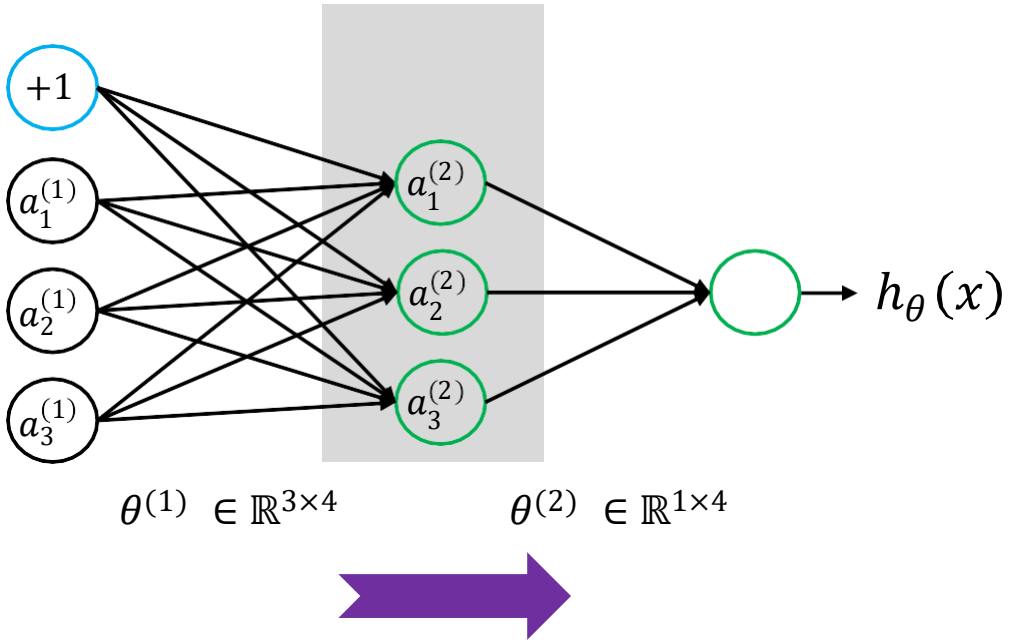


$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a_1^{(2)} = g(\theta_{10}^{(1)} a_0^{(1)} + \theta_{11}^{(1)} a_1^{(1)} + \theta_{12}^{(1)} a_2^{(1)} + \theta_{13}^{(1)} a_3^{(1)})$$
$$a_2^{(2)} = g(\theta_{20}^{(1)} a_0^{(1)} + \theta_{21}^{(1)} a_1^{(1)} + \theta_{22}^{(1)} a_2^{(1)} + \theta_{23}^{(1)} a_3^{(1)})$$
$$a_3^{(2)} = g(\theta_{30}^{(1)} a_0^{(1)} + \theta_{31}^{(1)} a_1^{(1)} + \theta_{32}^{(1)} a_2^{(1)} + \theta_{33}^{(1)} a_3^{(1)})$$

Gradient computation - forward propagation

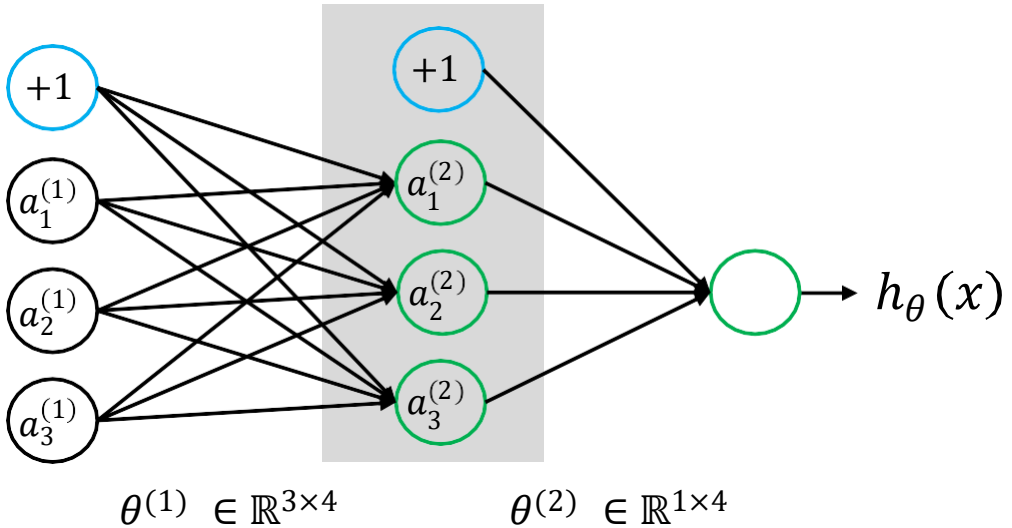
Layer 1 Layer 2 Layer 3



$$z^{(2)} = \theta^{(1)} a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$

$$a_1^{(2)} = g(\theta_{10}^{(1)} a_0^{(1)} + \theta_{11}^{(1)} a_1^{(1)} + \theta_{12}^{(1)} a_2^{(1)} + \theta_{13}^{(1)} a_3^{(1)})$$
$$a_2^{(2)} = g(\theta_{20}^{(1)} a_0^{(1)} + \theta_{21}^{(1)} a_1^{(1)} + \theta_{22}^{(1)} a_2^{(1)} + \theta_{23}^{(1)} a_3^{(1)})$$
$$a_3^{(2)} = g(\theta_{30}^{(1)} a_0^{(1)} + \theta_{31}^{(1)} a_1^{(1)} + \theta_{32}^{(1)} a_2^{(1)} + \theta_{33}^{(1)} a_3^{(1)})$$

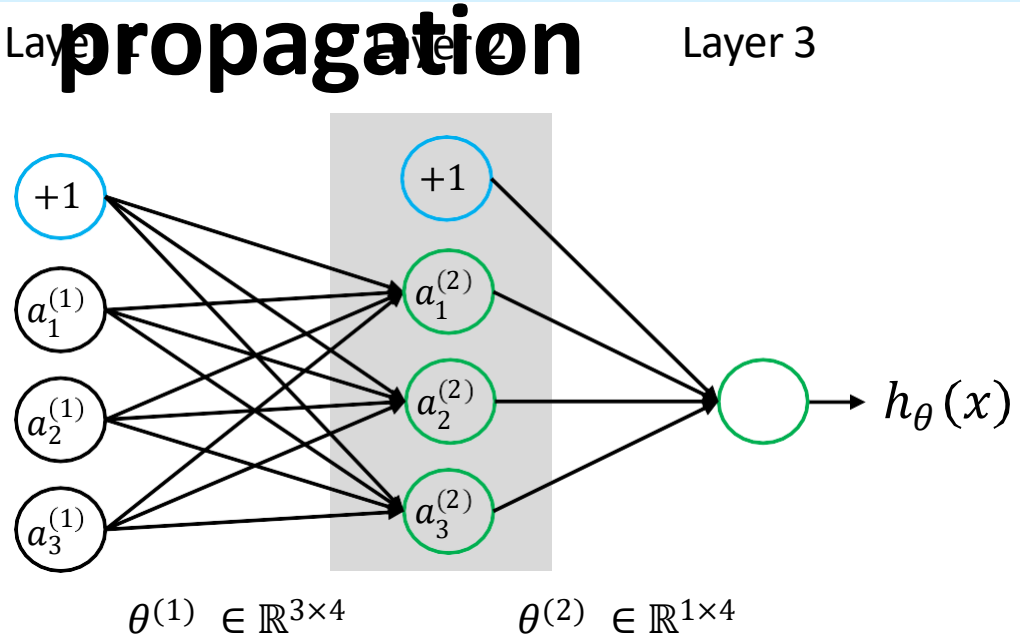
Gradient computation - forward propagation



$$z^{(2)} = \theta^{(1)} a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$
$$z^{(3)} = \theta^{(2)} a^{(2)}$$

$$a_1^{(2)} = g(\theta_{10}^{(1)} a_0^{(1)} + \theta_{11}^{(1)} a_1^{(1)} + \theta_{12}^{(1)} a_2^{(1)} + \theta_{13}^{(1)} a_3^{(1)})$$
$$a_2^{(2)} = g(\theta_{20}^{(1)} a_0^{(1)} + \theta_{21}^{(1)} a_1^{(1)} + \theta_{22}^{(1)} a_2^{(1)} + \theta_{23}^{(1)} a_3^{(1)})$$
$$a_3^{(2)} = g(\theta_{30}^{(1)} a_0^{(1)} + \theta_{31}^{(1)} a_1^{(1)} + \theta_{32}^{(1)} a_2^{(1)} + \theta_{33}^{(1)} a_3^{(1)})$$
$$a_1^{(3)} = g(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)})$$

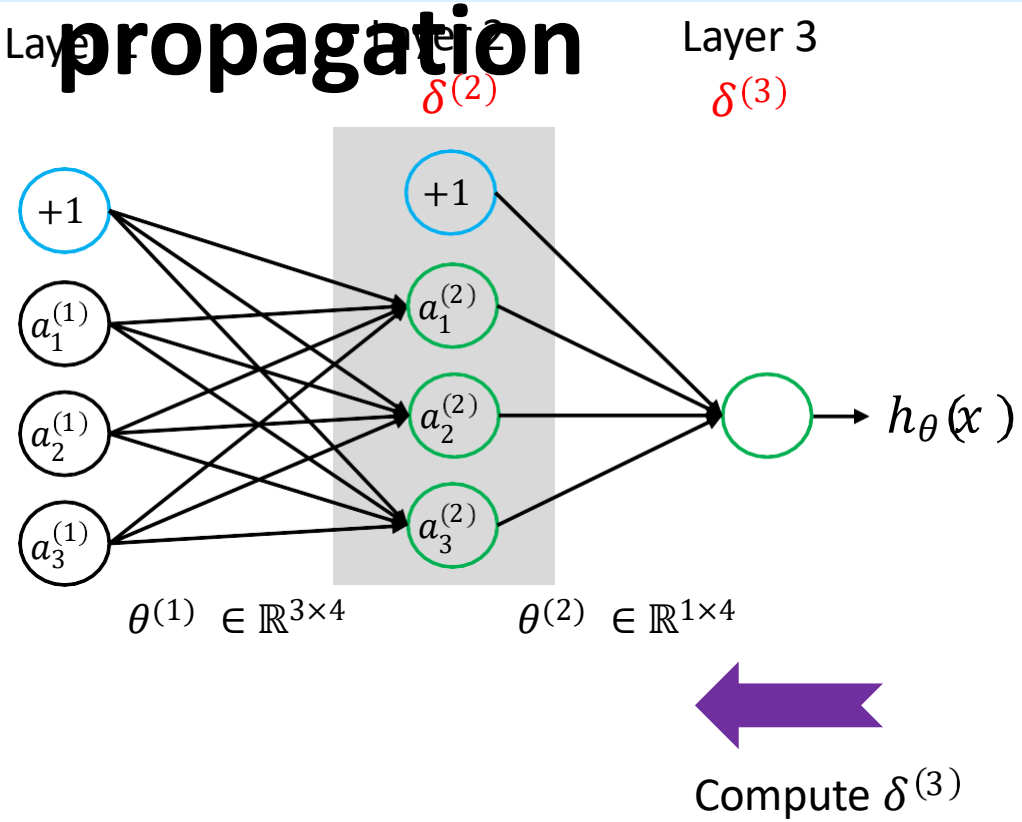
Gradient computation - forward propagation



$$z^{(2)} = \theta^{(1)}a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$
$$z^{(3)} = \theta^{(2)}a^{(2)}$$
$$a^{(3)} = g(z^{(3)})$$

$$a_1^{(2)} = g(\theta_{10}^{(1)}a_0^{(1)} + \theta_{11}^{(1)}a_1^{(1)} + \theta_{12}^{(1)}a_2^{(1)} + \theta_{13}^{(1)}a_3^{(1)})$$
$$a_2^{(2)} = g(\theta_{20}^{(1)}a_0^{(1)} + \theta_{21}^{(1)}a_1^{(1)} + \theta_{22}^{(1)}a_2^{(1)} + \theta_{23}^{(1)}a_3^{(1)})$$
$$a_3^{(2)} = g(\theta_{30}^{(1)}a_0^{(1)} + \theta_{31}^{(1)}a_1^{(1)} + \theta_{32}^{(1)}a_2^{(1)} + \theta_{33}^{(1)}a_3^{(1)})$$
$$a_1^{(3)} = g(\theta_{10}^{(2)}a_0^{(2)} + \theta_{11}^{(2)}a_1^{(2)} + \theta_{12}^{(2)}a_2^{(2)} + \theta_{13}^{(2)}a_3^{(2)})$$

Gradient computation - backward propagation

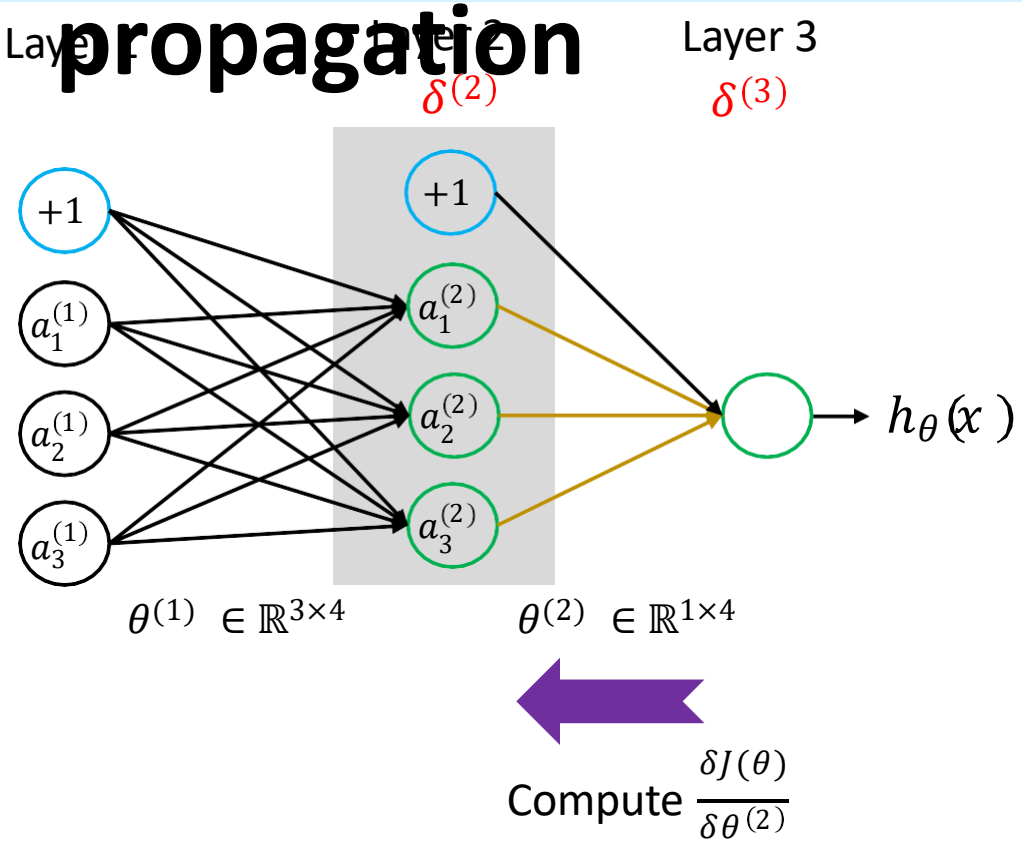


$\delta_j^{(l)}$ = error of j-th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$
$$z^{(3)} = \theta^{(2)} a^{(2)}$$
$$a^{(3)} = g(z^{(3)})$$
$$h_\theta(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}} = (a^{(3)} - y)\, g'(z^{(3)})$$

Gradient computation - backward propagation



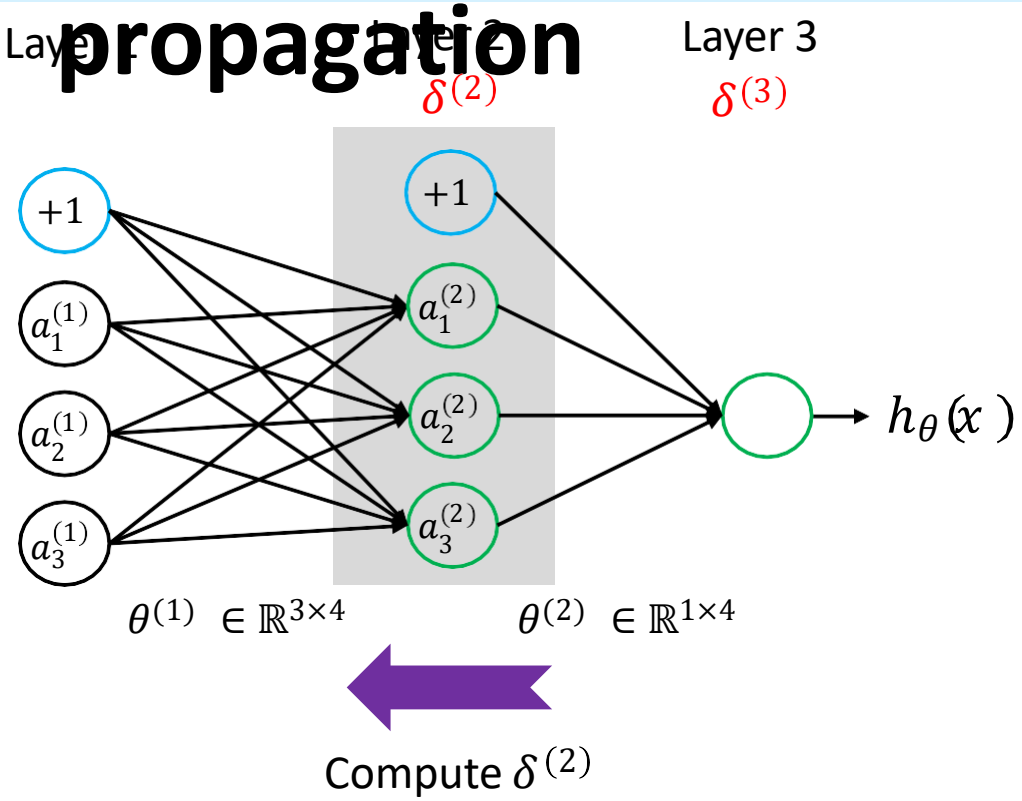
$\delta_j^{(l)}$ = error of j-th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$
$$z^{(3)} = \theta^{(2)} a^{(2)}$$
$$a^{(3)} = g(z^{(3)})$$
$$h_\theta(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}} = (a^{(3)} - y)\, g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

Gradient computation - backward propagation



$\delta_j^{(l)}$ = error of j-th neuron in layer l

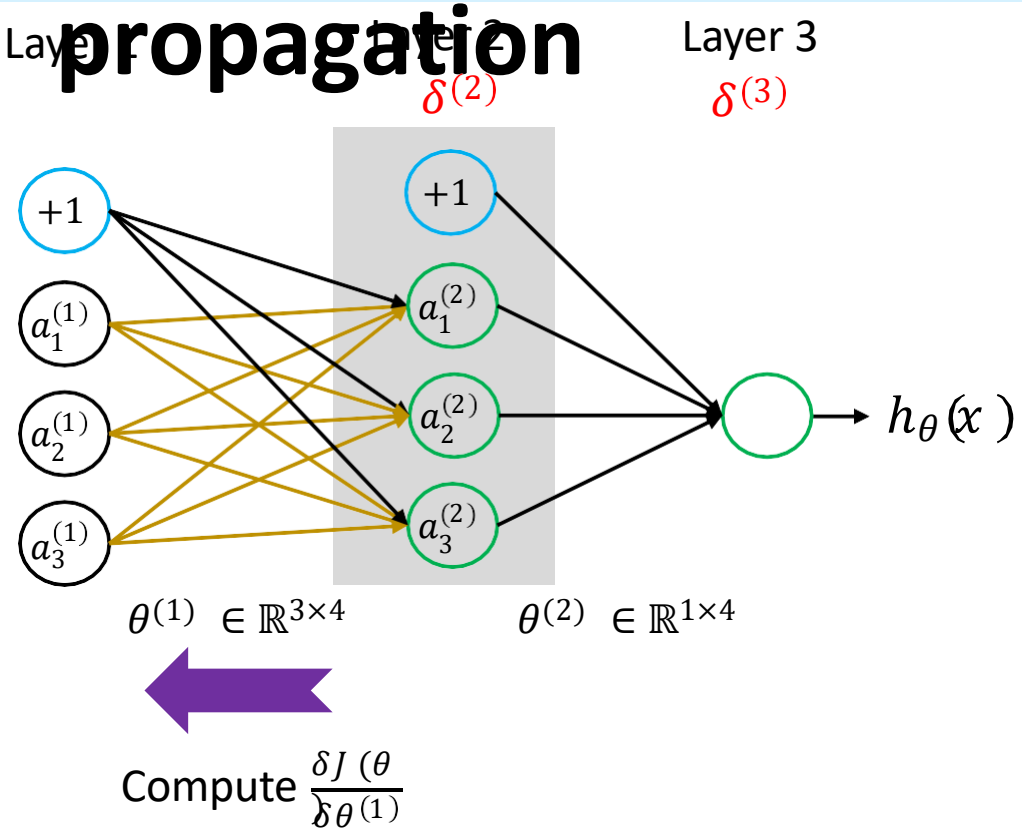
$$z^{(2)} = \theta^{(1)} a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$
$$z^{(3)} = \theta^{(2)} a^{(2)}$$
$$a^{(3)} = g(z^{(3)})$$
$$h_\theta(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}} = (a^{(3)} - y)\, g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

$$\delta^{(2)} = \frac{\delta J(\theta)}{\delta z^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta a^{(2)}} \frac{\delta a^{(2)}}{\delta z^{(2)}}$$
$$= \delta^{(3)} (\theta^{(2)})\, g'(z^{(2)})$$
$$= (\theta^{(2)})^{(T)} \delta^{(3)}\, g'(z^{(2)})$$

Gradient computation - backward propagation



$\delta_j^{(l)}$ = error of j-th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$
$$z^{(3)} = \theta^{(2)} a^{(2)}$$
$$a^{(3)} = g(z^{(3)})$$
$$h_\theta(x) = a^{(3)}$$

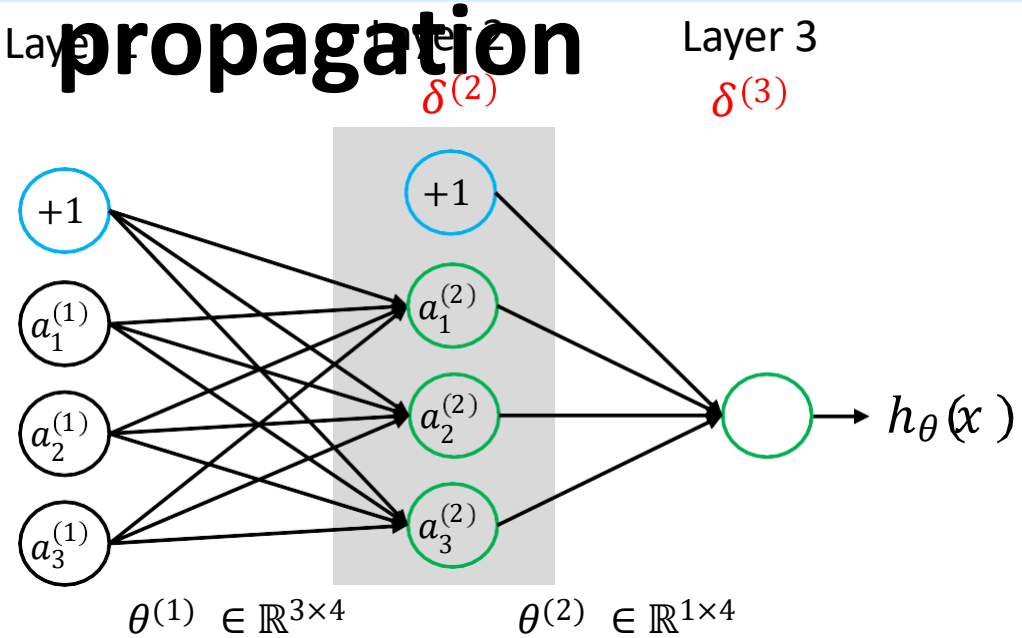
$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}} = (a^{(3)} - y)\, g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

$$\delta^{(2)} = \frac{\delta J(\theta)}{\delta z^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta a^{(2)}} \frac{\delta a^{(2)}}{\delta z^{(2)}}$$
$$= \delta^{(3)} (\theta^{(2)}) g'(z^{(2)})$$
$$= (\theta^{(2)})^{(T)} \delta^{(3)} \odot g'(z^{(2)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(1)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta a^{(2)}} \frac{\delta a^{(2)}}{\delta z^{(2)}} \frac{\delta z^{(2)}}{\delta \theta^{(1)}} = \delta^{(2)} a^{(1)}$$

Gradient computation - backward propagation



$\delta_j^{(l)}$ = error of j-th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$
$$a^{(2)} = g(z^{(2)})$$
$$z^{(3)} = \theta^{(2)} a^{(2)}$$
$$a^{(3)} = g(z^{(3)})$$
$$h_\theta(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}} = (a^{(3)} - y)\, g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

$$\delta^{(2)} = \frac{\delta J(\theta)}{\delta z^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta a^{(2)}} \frac{\delta a^{(2)}}{\delta z^{(2)}}$$
$$= \delta^{(3)} (\theta^{(2)}) g'(z^{(2)})$$
$$= (\theta^{(2)})^{(T)} \delta^{(3)} \odot g'(z^{(2)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(1)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta a^{(2)}} \frac{\delta a^{(2)}}{\delta z^{(2)}} \frac{\delta z^{(2)}}{\delta \theta^{(1)}} = \delta^{(2)} a^{(1)}$$

No $\delta^{(1)}$!

Gradients are computed using chain rule of differentiation

Summary of equation of backpropagation:

$$\delta^{(L)} = \frac{\delta J(\theta)}{\delta a^{(L)}} \frac{\delta a^{(L)}}{\delta z^{(L)}}$$

$$\delta^{(l)} = (\theta^{(l)})^{(T)} \delta^{(l+1)} \odot g'(z^{(l)})$$

$$\frac{\delta J(\theta)}{\delta \theta_j^{(l)}} = \delta_j^{(l+1)} a_k^{(l)}$$

$$\frac{\delta J(\theta)}{\delta \theta_j^{(l)}} = \delta_j^{(l+1)}$$

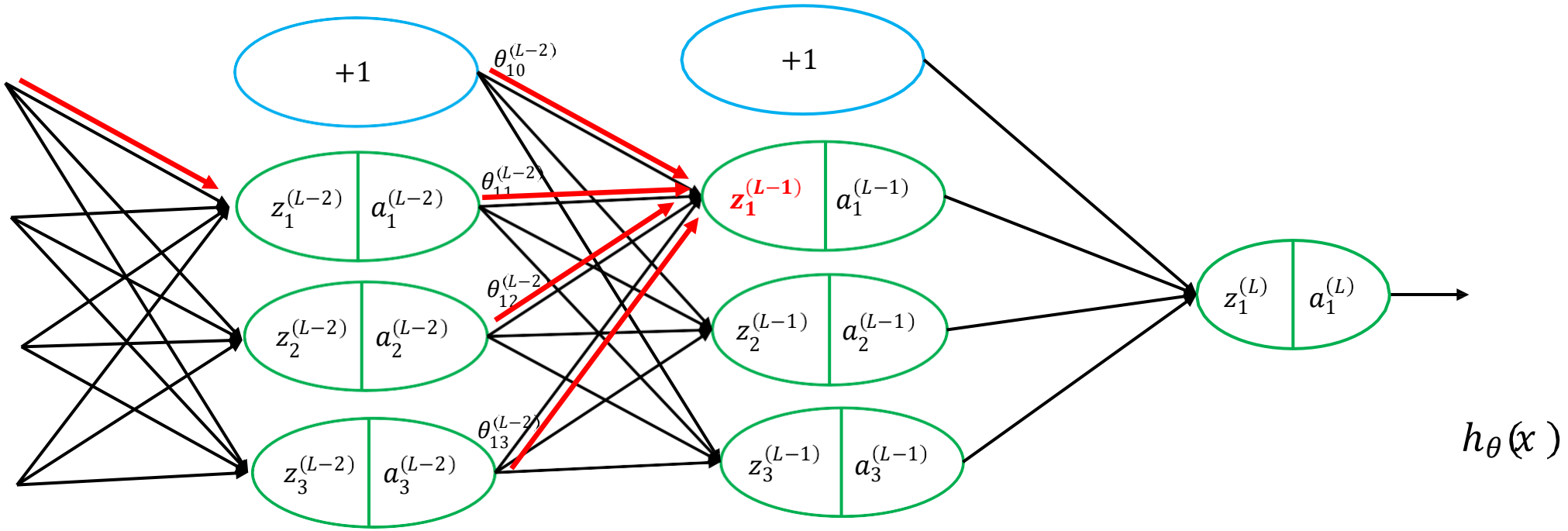
Backpropagation algorithm

1. **Input**: given *N* number of data points (training set) = $(x^{(1)}, y^{(1)})$, $(x^{(2)}, y^{(2)})$,……, $(x^{(N)}, y^{(N)})$
2. **Feedforward**: For each $l = 1,2, \ldots, L$, compute $z^{(l+1)} = \theta^{(l)} a^{(l)}$ and $a^{(l+1)} = g(z^{(l+1)})$
3. **Output error**: Compute $\delta^{(L)}$
4. **Backpropagate the error**: For each $l = 2, \ldots, L$, compute $\delta^{(l)}$
5. **Gradient computation**: Compute weights $\frac{\delta J(\theta)}{\delta \theta_{jk}^{(l)}}$ and biases $\frac{\delta J(\theta)}{\delta \theta_{j0}^{(l)}}$
6. **Optimization algorithm**: You can use any gradient descent optimization algorithm to minimize the error function $J(\theta)$.

Backpropagation intuition

- Forward propagation simply computes the weighted sum of $z^{(l)}$ from the left to the right.

E.g. $z^{(L-1)} = \theta_{10}^{(L-2)} a_0^{(L-2)} + \theta_{11}^{(L-2)} a_1^{(L-2)} + \theta_{12}^{(L-2)} a_2^{(L-2)} + \theta_{13}^{(L-2)} a_3^{(L-2)}$

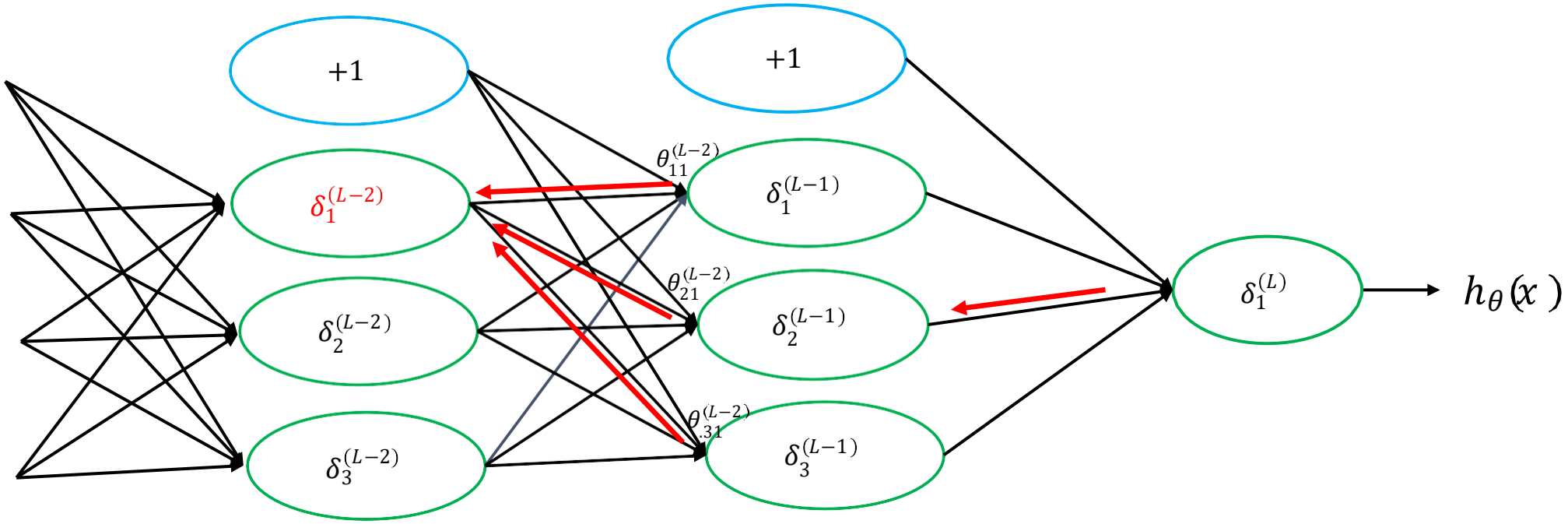


Backpropagation intuition

- Backward propagation computes the error term $\delta^{(l)}$ from the right to the left.

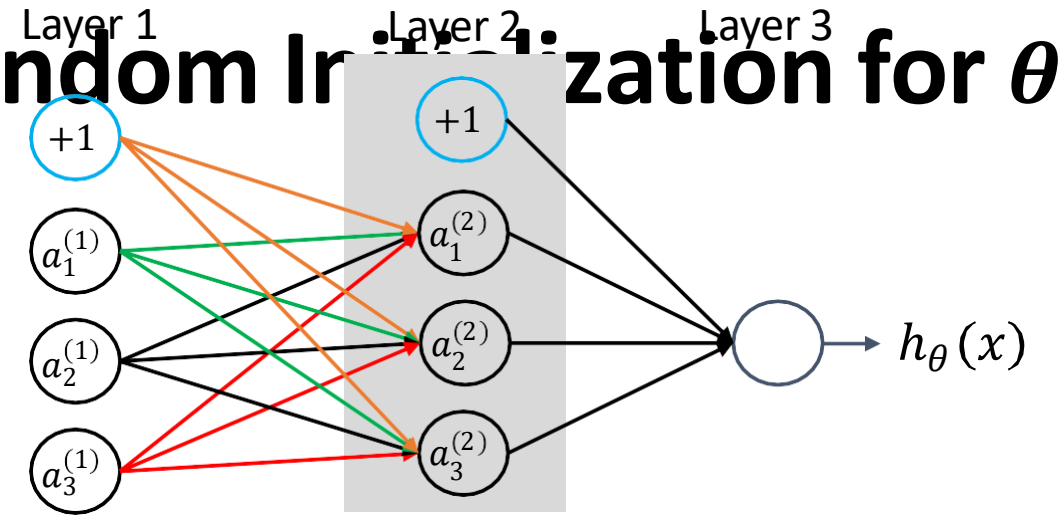
$$\text{E.g. } \delta_1^{(L-2)} = \theta_{11}^{(L-2)}\delta_1^{(L-1)} + \theta_{21}^{(L-2)}\delta_2^{(L-1)} + \theta_{31}^{(L-2)}\delta_3^{(L-1)}$$

- Measure of how much will we like to modify the weights $\theta_{jk}^{(l)}$ in order to affect this intermediate value of the calculation so as to influence the final result $h_\theta(x)$ and the cost function $J(\theta)$.



$( )$
jk

Random Initialization for $\theta_{jk}^{(l)}$

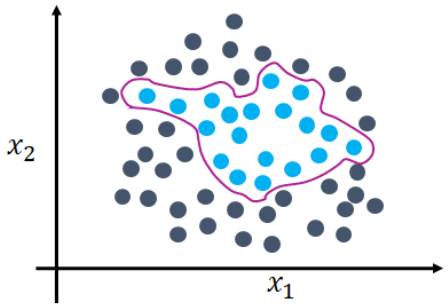


- By initializing the same weights for all layers, all hidden units calculate the same function.
- This is equivalent to learning the same features for each neuron in a hidden layer.
- To break the symmetry, initialize with random number (small value close to zero)

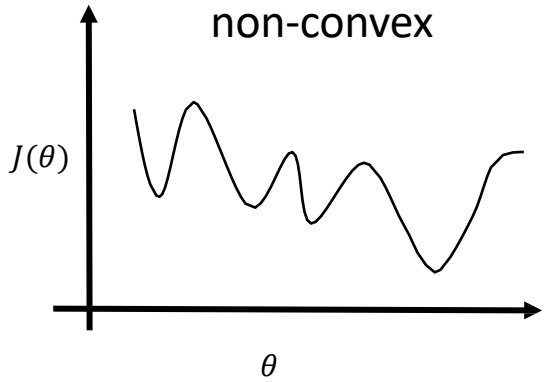
LR vs ANN

Neural Network

- Neural networks are much better for a **complex nonlinear hypothesis.**

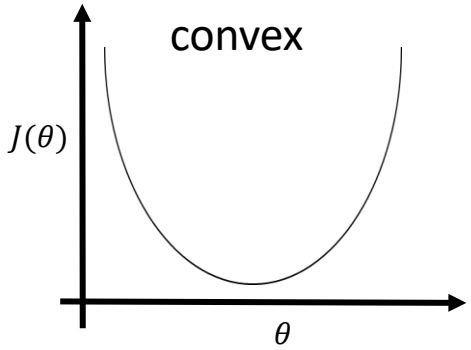


- A neural network is in general **not convex**. It can suffer from multiple local minima.



Logistic regression

- LR is useful for dataset where the classes are more or less "linearly separable"
- For "relatively" very **small** dataset.
- It is a great, robust model for **simple** classification tasks
- LR cost function (or max-entropy) is **convex**, and thus we are guaranteed to find the global cost minimum.



Next Lecture

❖Convolutional neural network